

Demonstrating DreamGarden: A Designer Assistant for Growing Games from a Single Prompt

Sam Earle

New York University
Brooklyn, New York, USA
Microsoft
Redmond, Washington, USA
smearle93@gmail.com

Samyak Parajuli

The University of Texas at Austin
Austin, Texas, USA
Microsoft
Redmond, Washington, USA
samyak.parajuli@utexas.edu

Andrzej Banburski-Fahey

Microsoft
Redmond, Washington, USA
abanburski@microsoft.com

Abstract

Coding assistants are increasingly leveraged in game design, both generating code and making high-level plans. To what degree can these tools align with developer workflows, and what new modes of human-computer interaction can emerge from their use? We present DreamGarden, an AI system capable of assisting with the development of diverse game environments in Unreal Engine. At the core of our method is an LLM-driven planner, capable of breaking down a single, high-level prompt—a dream, memory, or imagined scenario provided by a human user—into a hierarchical action plan, which is then distributed across specialized submodules facilitating concrete implementation. This system is presented to the user as a garden of plans and actions, both growing independently and responding to diverse forms of user intervention. Our GUI features a node-based tree editor in which users can provide seed prompts, prune and expand action plans, and provide feedback in real time.

CCS Concepts

• **Applied computing** → **Media arts**; • **Human-centered computing**;

Keywords

Game design assistants, 3D asset generation, large language models, visual feedback

ACM Reference Format:

Sam Earle, Samyak Parajuli, and Andrzej Banburski-Fahey. 2025. Demonstrating DreamGarden: A Designer Assistant for Growing Games from a Single Prompt. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3706599.3721282>

1 Introduction

A recent boom in generative AI has led to the widespread adoption of assistive AI tools by human creators, both casual and professional, in generating text, code, images and 3D assets. The AI models underpinning these tools are generally large neural networks, trained

on corpora of human-generated content scraped from the internet. Large language models (LLMs) are perhaps most ubiquitous, existing most commonly in the form of chatbots, predictive text models which have been fine-tuned via specialized datasets and human feedback to be generally “helpful” and excel in following user requests [10, 15, 28]. In the visual domain, diffusion models have brought on similarly impressive capabilities, allowing anyone to create visually convincing results from a single prompt [13, 20, 21].

Generally, these assistants turn individual user prompts into output wholesale, and users have little means of intervening in the process beyond asking the model to try again with adjusted prompts. Such a prompt-to-output paradigm becomes impractical as creative application domains become more complex. Designers of games or simulated environments (e.g. for film, architecture, or VR training) are unlikely to forego their domain expertise—and in particular their mastery of symbolic languages for describing artifacts—in favor of lengthy dialogues with fallible black box generators. Still, the latent expertise of large models holds great potential for assisting designers, and so a major hurdle in unlocking these capabilities is designing more suitable designer-assistant interaction paradigms.

Another obstacle toward leveraging assistants in complex domains is the sheer complexity of the tasks themselves. As the number of possibilities for error increases exponentially, the amount of back-and-forth that would be required between a user and the model in a straight-ahead prompting paradigm would result in intolerable user fatigue. It becomes necessary for the system to iterate autonomously to some degree to meet the user’s desiderata and work within their constraints, interfacing with design tools to evaluate and refine its own work without the need for constant user intervention and oversight.

The automated game design assistant we present in this work, *DreamGarden*, represents a possible answer to these questions. The core idea is to recursively prompt a large language model to devise a hierarchical plan of action, setting its own sub-goals which can be iterated on autonomously while interfacing with the design tools in question (in our case, Unreal Engine, a mesh generation diffusion model, and an asset database). This tackles the problem of complexity by subdividing tasks and allowing for autonomous iteration without requiring the user to act as a go-between for model output and the design tool. Meanwhile, the system’s plan and iteration attempts are exposed to users via a graphical interface, allowing them to intervene at a more granular level beyond the initial high-level prompt, but still at a higher level of abstraction than directly editing generated artifacts (though this is also a possibility within our system).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3721282>

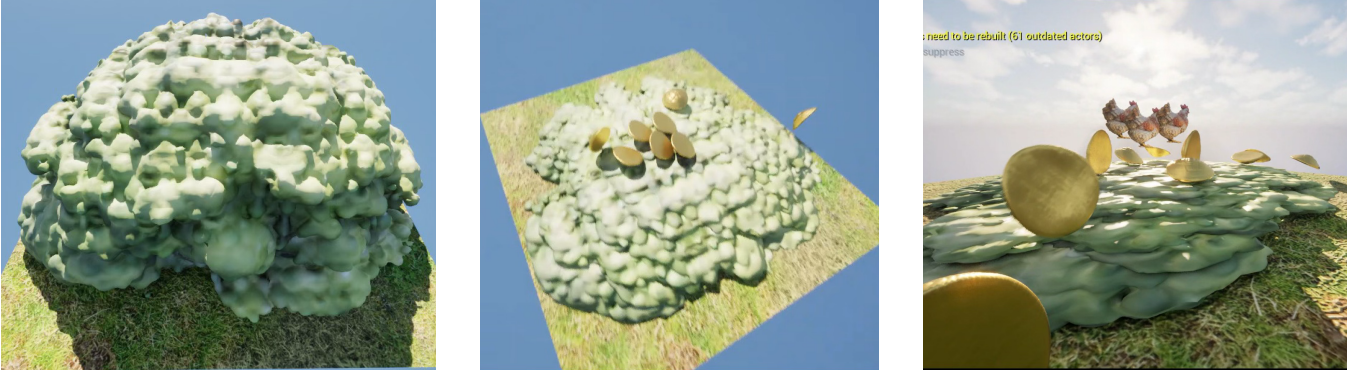


Figure 1: Example of autonomous scene-generation on DreamGarden. Here, the system iterates on a scene involving chickens laying golden eggs by writing and extending C++ Unreal Actor classes, beginning with some initial terrain and foliage, then adding dynamically-spawning and physics-enabled eggs and chickens using meshes that it has generated/downloaded at prior iterations.

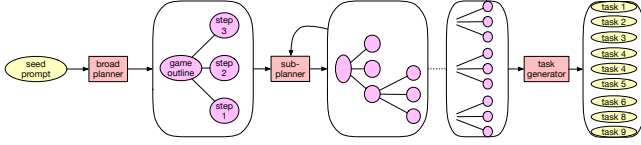


Figure 2: The planning module converts the user’s open-ended seed prompt into a broad plan for designing a simulation in Unreal Engine, then recursively breaks this plan down into more fine-grained steps, eventually terminating in leaf nodes which are then converted to concrete implementation tasks for the available implementation submodules.

2 Method

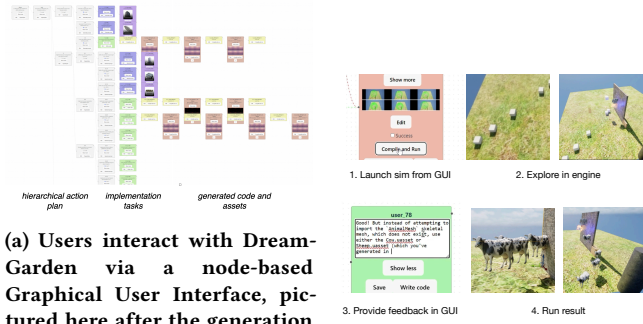
Planning module. The user inputs a seed prompt, which is opened and may describe a dream, an imagined scenario, or a rough game sketch. This is fed into the *broad planner*, which is system-prompted to take this open-ended input and reformulate it as a high-level plan for implementing the game. (All submodules/agents generate text and code using GPT-4o [16].) The *sub-planner* is then called recursively to grow this plan into a tree. This submodule is shown the entire plan tree in its current state, and asked to re-articulate a particular node (a step or sub-step of the plan) in more detail, then further expand it into a list of sub-steps. This process repeats, with the sub-planner processing nodes of the tree in breadth-first order, and flagging as “leaves” any nodes corresponding to concrete implementation steps. The leaf nodes are then translated into prompts or other formatted inputs for implementation submodules by the *task generator* (Figure 2). Once the *task generator* has transformed all leaf nodes into implementation tasks, these tasks are executed in sequence—i.e. visiting leaf sub-plan nodes in order—with the generated code, assets and layouts from prior tasks fed as input to subsequent submodules.

Code generator. The coding submodule involves the generation of C++ code for Unreal Engine “Actors”—essentially, general discrete entities within the simulation capable of arbitrary behavior

and interactions—and the generation of an initial json-structured layout for these actors in the level, which is then instantiated via a hard-coded python script inside the editor. (While *DreamGarden* is capable of creating single-player games, we mostly restrict it to 0-player simulations, so that it can provide self-contained feedback to itself without the need for a generalist game-playing agent, such as SIMA [18].) Results are fed back to the agent in the form of compilation and runtime logs, and screenshots of the first 6 seconds of in-engine simulation from a camera positioned to capture the initial bounding boxes of generated Actors. The agent may edit its output to address any issues (up to a fixed number of maximum attempts) or indicate success, at which point the system proceeds to the next implementation step. Code and layouts are cumulative, with the output of prior implementation steps injected into the prompts of coding agents at later steps.

Diffusion mesh generator. The diffusion mesh generation submodule generates novel 3D assets from scratch, by chaining together a text-to-2D diffusion-based image generator and a 2D-to-3D textured mesh generator. The task prompt for this submodule is a detailed description of the desired asset. The initial prompt is appended with some further directives in an attempt to make the image a suitable prompt for a 2D-3D model (i.e. requiring the object to be fully visible and posed against a blank background). This description is fed into a text-to-2D image generator, which we chose to be Dall-E 3 [14] in our experiments. The resulting image is then fed into a 2D-to-3D generator, in this case TripoSR [26], which we chose due to its speed and SOTA quality at the time of experiments, but this is modular and could be swapped to a completely different model.

Mesh downloader. For mesh download tasks, the task generator produces a concise description of the desired asset. This text is embedded via CLIP [19], and compared against embeddings of uploaded thumbnails of all assets in the Objaverse database [6]. The asset whose thumbnail has least CLIP distance to the text description is downloaded from the database. Meshes both generated and downloaded are saved as UE assets and exposed to subsequent coding agents for use in generated Actor code.



(a) Users interact with DreamGarden via a node-based Graphical User Interface, pictured here after the generation of the plan tree (left), implementation tasks (middle), and during iteration—involving code generation, in-editor simulation, automatic visual feedback, and code re-generation—on these implementation tasks by specialized submodules (right).

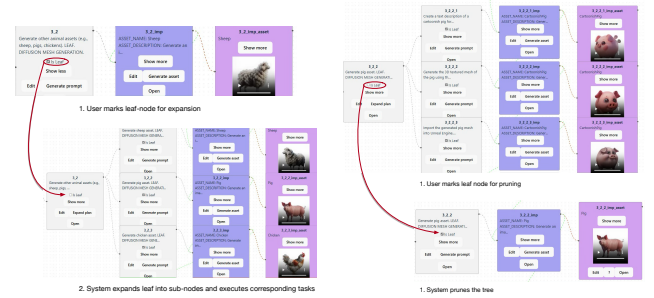
(b) Via the Graphical User Interface, users can compile and launch the generated simulation at various stages of development, exploring the result inside the Unreal Engine editor. They can then provide customized feedback to the code generator.

Figure 3: Users can seed the autonomous growth of a tree of plan and actions (left), and provide detailed feedback by engaging with generated simulations (right)

2.0.1 Graphical user interface. Because the action plan is effectively a tree, the basis of our GUI is a node-based editor, i.e. an interface in which users can manipulate nodes in a tree (Figure 3a). Nodes are either (sub-)steps in the hierarchical plan, implementation tasks, or steps taken by implementation submodules, and edges correspond to the hierarchical relationship between planning steps and tasks, and the chronological flow of implementation steps. We structure the backend call as a repeated function on a “frontier” of nodes to be expanded/implemented. The user’s edits can effectively result in pruning this tree, modifying nodes, and adding/removing them from this frontier.

The user begins with an empty canvas, and can add a seed node, where they enter the open-ended high level prompt. They then have the option to “step” or “play” the system, either stepping through a single call to the backend node-expansion function, or initiating the repeated expansion of nodes. (The step function is useful when the user intends to make a more active role, in that it prevents premature expansion of nodes whose ancestors might ultimately be edited by the user.) During expansion, child nodes fan out to the right of their parents. Plan nodes are represented in grey, and task nodes are color-coded according to the submodule to which they correspond. Special implementation submodule-specific attempt nodes contain video previews of downloaded/generated assets, or snippets of generated code, logs and visual previews (if applicable). Users can rearrange individual nodes or re-align the tree, and truncated node text can be expanded or opened in a separate window.

User edits. Each node in the action tree has an “is leaf” checkbox. By turning a non-leaf into a leaf node (i.e. when the system is unnecessarily granular), the user effectively prunes the tree. During the next call to the node-expansion function, any plan nodes that are children of this node are removed, as are any task nodes attached



(a) By unchecking the ‘Is Leaf’ option, a user can mark a given leaf node for further expansion by the sub-planner. Here the user marks for expansion a that erroneously called for the generation of multiple animal types via a single diffusion mesh generation task. Once the node is expanded, all listed animal types are generated via separate calls.

(b) By checking the ‘Is Leaf’ option, a user can mark a given leaf node for pruning. Here the user marks for pruning a node that calls for the generation of a single animal asset which has been superfluously expanded into multiple child nodes, resulting in redundant assets. After pruning, a single, standalone asset remains.

Figure 4: Users can prune or expand sub-trees by interacting with plan nodes in the GUI.

to these children. Similarly, if these tasks have already resulted in implementation step nodes, then these implementation nodes are also removed (this data is backed up to a separate folder, should the user want to revert their changes). Finally, because any code and assets resulting from these implementation steps would have been handed down to implementation tasks following it in the ordered sequence of leaf nodes, then any implementation steps belonging to these tasks are also backed up and removed from the current tree.

Conversely, by turning a leaf node into a non-leaf node, the user effectively adds this node to the frontier, queuing it for expansion into a sub-plan. If a task node was generated for this leaf, it is backed up and removed from the tree along with any implementation attempt nodes belonging to this task, or any implementation nodes belonging to tasks following it in the ordered list of tasks.

During code generation, coding attempts/evaluations are displayed in nodes that are chained in a sequence, moving from left to right out from the parent task node (itself stepping from a leaf node of the plan tree). When the generated code has compiled and run successfully in UE, the corresponding attempt node will contain a visual preview of the simulation as well as a “compile & run” button, which allows the user to compile the code as it was at this state and launch the editor with the corresponding actor layout. The user is then able to inspect the resultant simulation in UE. They may make edits to generated code (testing them via live recompilation from within the editor), which when saved would then be fed into subsequent iterations of code generation. They may also return to the node-based GUI and attach a new manual feedback node to the attempt node in question. This will result in the pruning of all implementation attempts descending from this node (i.e. to the right of or below it).

3 Conclusion & Future Work

DreamGarden demonstrates the promise of interfacing multi-agent LLM-driven systems with game engines for open-ended content creation. The modular nature of generated plan and action trees opens the door to yet more forms of user interaction beyond prompting, pruning and expansion. Future work could generate (or solicit from users) multi-modal sketches (e.g. rough images of levels, diagrams of mechanics, or toy 2D prototype games) which could be used to guide the later generation of the final experience in Unreal Engine, or incorporate additional auxiliary tools (such as Blender [2] or Houdini [24]) for specialized content generation via implementation submodules.

Meanwhile, the recursive growth of a tree of plans that to be implemented by predefined implementation submodules is a promising and general approach to multi-agent orchestration in complex domains. Future work could investigate fine-tuning LLMs on proven high level-implementation plans of existing projects, both in the gaming industry and in software development more broadly.

In preliminary usability studies (reported in concurrent work), we note the strong appeal among users of a branching approach to “growing” content collaboratively with AI agents, and argue that a graph-based approach is a natural fit for creative and open-ended tasks. We remark that the paradigm of “gardening”—as opposed to constant active human guidance or a fully passive “prompt and sit back” approach—invites a variety of modes of interaction from users with differing intents and levels of experience. The key here is in combining autonomous feedback and self-improvement with a variety of means of user intervention. This is strengthened by our focus on having the system operate on human-interpretable representations of generated content inside an established game engine, in contrast to extant efforts that seek to replace game engines with effectively uninterpretable latent variables [3].

4 Related work

DreamGarden is both a designer-facing tool and a self-perpetuating ecosystem of design ideas and content. In the latter regard, we take inspiration from the field of Artificial Life [1], which includes the design of simulated environments that foster the emergence of complex and life-like phenomena. If such autonomous novelty-seeking systems can be likened to simplistic digital petri dishes or ecosystems [7], then when augmented with a user-facing aspect they can be thought of as gardens. Often, these gardens are communal. [4] develop an online Twitterbot which seeks to autonomously generate aesthetically pleasing gridworld game maps by soliciting human feedback via automated polls. PicBreeder [23] solicits user participation in interactive evolution, resulting in the discovery of novel designs without users setting these discoveries as explicit objectives in advance. Though *DreamGarden* is framed as a system for translating a prompt to a corresponding end product, the plans and artifacts it sometimes generates could potentially be explored as art objects on their own merits, with the system’s intermediary outputs repurposed for divergent, open-ended search [25].

Prior work has explored the use of language models to generate games from scratch. Word2World [11] generates narratives from scratch and subsequently uses this narrative to specify a set of

characters, a tileset, and a sequence of grid-based level layouts. Individual tiles are then generated using the text-to-image model Stable Diffusion [21]. Here, game mechanics and objectives are essentially fixed in advance, and limited to player movement, the impassibility of certain tile types, and achievement of certain objectives by reaching special tiles. By contrast, *DreamGarden* is unrestricted in game mechanics—in terms of interactions between abstract Actor classes written in C++—which can be determined from scratch by our code generation submodule.

The procedural generation of entire games has been an area of research prior to the rise of modern deep learning, with the first experiments in Automatic Game Design demonstrated in [12] and [27]. The prospect of learning game design directly from games was proposed in [17]. Recombining learned features for game design was considered in [8], while in [22] authors have shown how blending of different games can be achieved using ML techniques. For overviews of the field, see [9, 29]. The authors of [5] raised pressing questions around the valuation of creativity and the participation of autonomous agents in game development communities. Though *DreamGarden* pushes the frontier of automatic game generation forward by dynamically orchestrating specialized models, it is built on the assumption that deep human engagement is required to generate socially meaningful artifacts.

References

- [1] Wendy Aguilar, Guillermo Santamaria-Bonfil, Tom Froese, and Carlos Gershenson. 2014. The past, present, and future of artificial life. *Frontiers in Robotics and AI* 1 (2014), 8.
- [2] Blender Foundation. [n. d.]. *Blender software*. <https://www.blender.org/>
- [3] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. 2024. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*.
- [4] Megan Charity and Julian Togelius. 2022. Aesthetic bot: interactively evolving game maps on twitter. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 18. 18–25.
- [5] Michael Cook, Simon Colton, and Jeremy Gow. 2016. The angelina videogame design system—part i. *IEEE Transactions on Computational Intelligence and AI in Games* 9, 2 (2016), 192–203.
- [6] Matt Deitke, Ruoshi Liu, Matthew Wallingford, Huong Ngo, Oscar Michel, Aditya Kusupati, Alan Fan, Christian Laforte, Vikram Voleti, Samir Yitzhak Gadre, et al. 2024. Objaverse-xl: A universe of 10m+ 3d objects. *Advances in Neural Information Processing Systems* 36 (2024).
- [7] Maxence Faldor and Antoine Cully. 2024. Toward Artificial Open-Ended Evolution within Lenia using Quality-Diversity. *arXiv preprint arXiv:2406.04235* (2024).
- [8] Matthew Guzdial and Mark Riedl. 2018. Automated game design via conceptual expansion. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 14. 31–37.
- [9] Mark Hendrikx, Sebastiaan Meijer, Joeri Van Der Velden, and Alexandru Iosup. 2013. Procedural content generation for games: A survey. *ACM Trans. Multimedia Comput. Commun. Appl.* 9, 1, Article 1 (Feb. 2013), 22 pages. doi:10.1145/2422956.2422957
- [10] Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L’elio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *ArXiv abs/2310.06825* (2023). <https://api.semanticscholar.org/CorpusID:263830494>
- [11] Muhammad U Nasir, Steven James, and Julian Togelius. 2024. Word2World: Generating Stories and Worlds through Large Language Models. *arXiv preprint arXiv:2405.06686* (2024).
- [12] Mark J. Nelson and Michael Mateas. 2007. Towards Automated Game Design. In *AI’IA 2007: Artificial Intelligence and Human-Oriented Computing*, Roberto Basili and Maria Teresa Pazienza (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 626–637.

- [13] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. 2021. GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models. In *International Conference on Machine Learning*. <https://api.semanticscholar.org/CorpusID:245335086>
- [14] OpenAI. 2023. DALL-E 3. <https://openai.com/dall-e-3> A text-to-image generation system developed by OpenAI, accessed via ChatGPT Plus and Enterprise, and through the OpenAI API.
- [15] OpenAI. 2024. ChatGPT: Language Model. <https://chat.openai.com/>. Accessed: 2024-09-02.
- [16] OpenAI. 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>. Accessed: 2024-09-11.
- [17] Joseph C Osborn, Adam Summerville, and Michael Mateas. 2017. Automated game design learning. In *2017 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 240–247.
- [18] Maria Abi Raad, Arun Ahuja, Catarina Barros, Frederic Besse, Andrew Bolt, Adrian Bolton, Bethanie Brownfield, Gavin Buttimore, Max Cant, Sarah Chakera, et al. 2024. Scaling instructable agents across many simulated worlds. *arXiv preprint arXiv:2404.10179* (2024).
- [19] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [20] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical Text-Conditional Image Generation with CLIP Latents. *ArXiv abs/2204.06125* (2022). <https://api.semanticscholar.org/CorpusID:248097655>
- [21] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10684–10695.
- [22] Anurag Sarkar and Seth Cooper. 2020. Towards Game Design via Creative Machine Learning (GDCML). In *2020 IEEE Conference on Games (CoG)*. 744–751. doi:10.1109/CoG47356.2020.9231927
- [23] Jimmy Secretan, Nicholas Beato, David B D'Ambrosio, Adelein Rodriguez, Adam Campbell, Jeremiah T Folsom-Kovarik, and Kenneth O Stanley. 2011. Picbreeder: A case study in collaborative evolutionary exploration of design space. *Evolutionary computation* 19, 3 (2011), 373–403.
- [24] SideFX. [n. d.]. *Houdini software*. <https://www.sidefx.com/products/houdini/>
- [25] Kenneth O Stanley and Joel Lehman. 2015. *Why greatness cannot be planned: The myth of the objective*. Springer.
- [26] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. 2024. Triposr: Fast 3d object reconstruction from a single image. *arXiv preprint arXiv:2403.02151* (2024).
- [27] Julian Togelius and Jurgen Schmidhuber. 2008. An experiment in automatic game design. In *2008 IEEE Symposium On Computational Intelligence and Games*. 111–118. doi:10.1109/CIG.2008.5035629
- [28] Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrutl Bhoale, Daniel M. Bikel, Lukas Blecher, Cristian Cantón Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *ArXiv abs/2307.09288* (2023). <https://api.semanticscholar.org/CorpusID:259950998>
- [29] Georgios N Yannakakis and Julian Togelius. 2018. *Artificial intelligence and games*. Vol. 2. Springer.